

Script - controlled ACL - Restrict Record Access Based On Field Value

Project Documentation

Team ID: SWTID-2026-4968

Date: 29 September 2026

1. Introduction

- Team Members: 5

S.No	Team Member Name	Role
1	Janani .S	Team Leader
2	Biruntha.S	Team Member
3	Sripriya.M	Team Member
4	Muthulakshmi.M	Team Member
5	Nishamerlin.J	Team Member

2. Project Overview

Purpose

The purpose of this project is to build a MERN stack application that protects records using a script-controlled Access Control List (ACL). Instead of giving access only by role, each request is evaluated by a server-side script that checks the value of a field in the record (for example *department*, *status* or *confidentiality*) against the logged-in user. Records are shown, edited or hidden only when the script returns true. The goal is to restrict record access based on field value in a secure and flexible way.

Features

- Script-controlled ACL rules written in JavaScript and evaluated on the server for every request.
- Record-level access restriction based on field value (e.g. department, status, confidentiality level).
- Separate permissions for read, create, write and delete operations.
- Field-level protection: restricted fields are masked or removed from the API response.
- User registration and login with JWT authentication and role support (Admin, Manager, Employee).
- Admin screen to create, edit, enable or disable ACL rules and scripts without changing code.
- Audit log that records every allowed and denied access attempt.
- Responsive React user interface that shows only the records the user is allowed to see.

3. Architecture

Frontend

The frontend is built with React. Pages include Login, Register, Dashboard, Records List, Record Details and the ACL Rules admin page. React Router handles navigation, Axios sends API requests with the JWT token, and Context API stores the logged-in user. Protected routes redirect unauthenticated users to the login page. The UI renders only the data returned by the backend, so restricted records never reach the browser.

Backend

The backend uses Node.js and Express.js in a layered structure: routes, controllers, middleware and models. An authentication middleware verifies the JWT. An ACL middleware then loads the active ACL rules for the requested table and operation and runs each rule script inside a sandbox (Node.js *vm* module) with the objects *current* (the record) and *user*. If the script sets *answer = true*, access is granted; otherwise the API returns 403 Forbidden. For list requests, the same script is applied to filter out records the user cannot see.

Database

MongoDB stores data using Mongoose schemas.

Collection	Key Fields	Description
users	name, email, password (hashed), role, department	Application users and their attributes
records	title, description, department, status, confidentiality, createdBy, timestamps	Business records protected by ACL
acl_rules	name, table, operation, field, script, active	Script-controlled ACL rules
audit_logs	user, recordId, operation, result, timestamp	History of allowed and denied access

Example ACL script

```
// Operation: read | Table: records
// Allow access only when the record belongs to the user's department
// or the confidentiality level is 'public'
answer = false;
if (user.role === 'admin') {
  answer = true;
} else if (current.confidentiality === 'public') {
  answer = true;
} else if (current.department === user.department) {
  answer = true;
}
```

4. Setup Instructions

Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local installation or MongoDB Atlas)
- Git
- A code editor such as VS Code
- Postman (optional, for API testing)

Installation

- Clone the repository:

```
git clone <repository-url>
cd <project-folder>
```

- Install dependencies for both client and server:

```
cd client
npm install
cd ../server
npm install
```

- Create a **.env** file inside the server folder:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/script_acl
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=1d
```

- (Optional) Seed the database with sample users, records and ACL rules:

```
cd server
npm run seed
```

5. Folder Structure

Client

```
client/
  public/
  src/
    components/      # Navbar, RecordCard, RuleForm, ProtectedRoute
    pages/           # Login, Register, Dashboard, Records, AclRules
    context/         # AuthContext
    services/        # Axios API calls
    App.js
    index.js
```

Server

```
server/
  config/            # Database connection
  controllers/       # authController, recordController, aclController
  middleware/        # authMiddleware, aclMiddleware
  models/           # User, Record, AclRule, AuditLog
  routes/           # authRoutes, recordRoutes, aclRoutes
  utils/            # scriptRunner (sandbox), auditLogger
  server.js
  .env
```

6. Running the Application

Start both servers locally:

- **Frontend:** `npm start` in the client directory (runs on `http://localhost:3000`).

```
cd client
npm start
```

- **Backend:** `npm start` in the server directory (runs on `http://localhost:5000`).

```
cd server
npm start
```

7. API Documentation

Base URL: `http://localhost:5000/api`. All endpoints except register and login need the header `Authorization: Bearer <token>`.

Method	Endpoint	Description	Access
POST	<code>/auth/register</code>	Register a new user	Public
POST	<code>/auth/login</code>	Login and receive JWT	Public
GET	<code>/records</code>	List records allowed by ACL	Authenticated
GET	<code>/records/:id</code>	Get one record (read ACL)	Authenticated
POST	<code>/records</code>	Create record (create ACL)	Authenticated
PUT	<code>/records/:id</code>	Update record (write ACL)	Authenticated
DELETE	<code>/records/:id</code>	Delete record (delete ACL)	Authenticated
GET	<code>/acl</code>	List ACL rules	Admin
POST	<code>/acl</code>	Create ACL rule with script	Admin
PUT	<code>/acl/:id</code>	Update ACL rule	Admin
DELETE	<code>/acl/:id</code>	Delete ACL rule	Admin

Example: POST `/auth/login`

Request body:

```
{ "email": "user@example.com", "password": "123456" }
```

Response 200:

```
{ "token": "<jwt-token>",  
  "user": { "id": "665f...", "name": "Arun", "role": "employee", "department": "IT" } }
```

Example: GET `/records/:id`

Response 200 (ACL script returned true):

```
{ "_id": "6660...", "title": "Server Upgrade", "department": "IT",  
  "status": "open", "confidentiality": "internal" }
```

Response 403 (ACL script returned false):

```
{ "message": "Access denied by ACL rule" }
```

Example: POST `/acl`

Request body:

```
{ "name": "Read own department records", "table": "records", "operation": "read",  
  "script": "answer = (current.department === user.department);", "active": true }
```

Response 201:

```
{ "message": "ACL rule created", "id": "6661..." }
```

8. Authentication

Authentication and authorization are handled as follows:

- **Password security:** Passwords are hashed with bcrypt before being stored in MongoDB.
- **Tokens:** After a successful login the server issues a signed JSON Web Token (JWT) containing the user id and role. The client stores it and sends it in the Authorization header with every request.
- **Verification:** The authentication middleware verifies the token and attaches the user to the request. Invalid or expired tokens return 401 Unauthorized.

- **Authorization (Script - controlled ACL):** After authentication, the ACL middleware finds the active rules for the table and operation and runs the script in a sandbox. Access is allowed only if the script sets *answer* to true; otherwise the API returns 403 Forbidden.
- **Role checks:** Admin-only routes (ACL rule management) are additionally protected by a role-check middleware.
- **Default deny:** If no rule matches or a script throws an error, access is denied.
- **Auditing:** Every allowed or denied decision is saved in the `audit_logs` collection.

9. User Interface

Screenshots or GIFs of the following UI features should be added here:

- Login and Registration pages
- Dashboard showing only the records allowed for the user
- Record list filtered by field value (department / status)
- Record details page with masked restricted fields
- Access denied message for a restricted record
- ACL Rules admin page with the script editor
- Audit log page

[Insert screenshots / GIFs here]

10. Testing

The testing strategy covers unit, API and manual UI testing.

- **Unit testing:** Jest to test the script runner and ACL evaluation logic (allow, deny and error cases).
- **API testing:** Postman and Supertest to verify status codes (200, 401, 403) for users with different departments and roles.
- **Frontend testing:** React Testing Library for components and protected routes.
- **Manual testing:** Login with different users and confirm that each sees only the records matching the ACL field-value rules.
- **Negative testing:** Try invalid tokens, unauthorized record ids and malformed scripts.

Test Case	Input	Expected Result
Same department user reads record	<code>user.department = IT,</code> <code>record.department = IT</code>	200 OK - record returned
Different department user reads record	<code>user.department = HR,</code> <code>record.department = IT</code>	403 Forbidden
Public record read by any user	<code>record.confidentiality = public</code>	200 OK - record returned
Request without token	No Authorization header	401 Unauthorized
Script with error	Rule script throws exception	Access denied and error logged

11. Screenshots or Demo

[Insert screenshots here]

Demo link: *[Add your demo video / deployment link here]*

12. Known Issues

- Complex or long-running ACL scripts can slow down list requests because each record is evaluated.
- Only JavaScript scripts are supported inside the sandbox.
- ACL rule changes may need a page refresh before the UI reflects them.
- No pagination on the audit log page for very large data sets.

13. Future Enhancements

- Add a visual (no-code) rule builder alongside the script editor.
- Support rule priority ordering and conflict detection.
- Cache evaluated rules using Redis to improve performance.
- Add multi-factor authentication and refresh tokens.
- Provide export and analytics of audit logs.
- Add role and group hierarchy and inheritance of permissions.